Tasks for Prospective GSoC 2026 Applicants for the

SPINN Project

Submission Guidelines

- **IN ORDER TO BE CONSIDERED AS AN APPLICANT FOR GOOGLE SUMMER OF CODE YOU MUST ALSO SUBMIT A PROPOSAL THROUGH THE GOOGLE SUMMER OF CODE PORTAL**
- Additional tasks are listed below
- When completed, please use [this Google form]() to submit

Demonstrate proficiency with Physics-Informed Neural Networks (PINNs) by selecting a physics problem and training a PINN to accurately solve the associated boundary-value problem.

Possible example tasks include (but are not limited to):

(i) solving the Poisson equation on a 2D domain with user-specified boundary conditions;

(ii) solving the steady-state incompressible Navier–Stokes equations for flow around a specified geometry;

(iii) solving a 1D boundary-value ODE with a user-chosen forcing term.

If you have difficulty converging to a correct solution, document likely causes (e.g., scaling, sampling strategy, loss-weighting, boundary-condition enforcement) and propose concrete steps to improve convergence. Please include relevant visualizations (loss curves, PINN solution plots, etc.) that help convey the results and demonstrate your understanding. It may be helpful to choose an example problem with a well known analytical solution for more helpful comparisons with the PINN output. Please provide access to your code and include clear, concise comments explaining the key components and overall approach.

$$\frac{d^2x}{dz^2} + 2\xi\frac{dx}{dz} + x = 0\,, \quad z\epsilon[0, 20], \quad \xi\epsilon[0.1,\ 0.4]$$

I use this equation for damped harmonic oscillator:

$$\frac{d^2x}{dz^2} + 2\xi\frac{dx}{dz} + x = 0, \quad z\epsilon[0, 20], \quad \xi\epsilon[0.1,\ 0.4]$$

Here: $x(0, \xi) = x_0 = 0.7, \ \frac{dx}{dz}(0,\ \xi) = v_0 = 1.2$

PINN will take $(z,\ \xi)$ as input and output of $x(z,\ \xi)$

Imports and basic configuration:

```
[3]: import torch
     import torch.nn as nn
     import torch.optim as optim

     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

     # Problem constants
     x0 = 0.7
     v0 = 1.2
     z_min, z_max = 0.0, 20.0
     xi_min, xi_max = 0.1, 0.4
```

Define the PINN architecture
We build a simple fully connected network that maps $(z,\xi)\rightarrow x$.

```
[4]: class PINN(nn.Module):
         def __init__(self, in_dim=2, out_dim=1, hidden_dim=64, num_hidden_layers=4):
             super().__init__()
             layers = []
             layers.append(nn.Linear(in_dim, hidden_dim))
             layers.append(nn.Tanh())
             for _ in range(num_hidden_layers - 1):
                 layers.append(nn.Linear(hidden_dim, hidden_dim))
                 layers.append(nn.Tanh())
             layers.append(nn.Linear(hidden_dim, out_dim))
             self.net = nn.Sequential(*layers)

         def forward(self, z, xi):
             inp = torch.cat([z, xi], dim=1)
             return self.net(inp)
```

Instantiate the model:

```
[5]: model = PINN().to(device)
```

Sampling collocation and initial-condition points

- Collocation points in the interior of the domain for the ODE residual.
- Initial-condition points at z=0 for all ξ in the range.

```
[6]: def sample_collocation_points(N_f):
         z_f = torch.rand(N_f, 1) * (z_max - z_min) + z_min
         xi_f = torch.rand(N_f, 1) * (xi_max - xi_min) + xi_min
         return z_f.to(device), xi_f.to(device)

     def sample_initial_points(N_ic):
         z_ic = torch.zeros(N_ic, 1)
         xi_ic = torch.rand(N_ic, 1) * (xi_max - xi_min) + xi_min
         return z_ic.to(device), xi_ic.to(device)
```

Compute derivatives with autograd

We need xz and xzz with respect to z.

```
[7]: def derivatives(model, z, xi):
         z.requires_grad_(True)
         xi.requires_grad_(True)

         x = model(z, xi)

         dx_dz = torch.autograd.grad(
             x, z,
             grad_outputs=torch.ones_like(x),
             retain_graph=True,
             create_graph=True
         )[0]

         d2x_dz2 = torch.autograd.grad(
             dx_dz, z,
             grad_outputs=torch.ones_like(dx_dz),
             retain_graph=True,
             create_graph=True
         )[0]

         return x, dx_dz, d2x_dz2
```

Define the physics and initial-condition losses

- **Physics residual**:

r(z,ξ)=xzz+2ξxz+x

- **Initial conditions** at z=0:

x(0,ξ)−x0=0,xz(0,ξ)−v0=0

```
[8]: def loss_function(model, N_f=2000, N_ic=200):
         # Collocation points
         z_f, xi_f = sample_collocation_points(N_f)
         x_f, dx_dz_f, d2x_dz2_f = derivatives(model, z_f, xi_f)

         # Physics residual
         residual = d2x_dz2_f + 2.0 * xi_f * dx_dz_f + x_f
         loss_f = torch.mean(residual**2)

         # Initial conditions
         z_ic, xi_ic = sample_initial_points(N_ic)
         x_ic, dx_dz_ic, _ = derivatives(model, z_ic, xi_ic)

         loss_x0 = torch.mean((x_ic - x0)**2)
         loss_v0 = torch.mean((dx_dz_ic - v0)**2)

         loss_ic = loss_x0 + loss_v0
         total_loss = loss_f + loss_ic

         return total_loss, loss_f, loss_ic
```

Training loop

You can later fine-tune:

- `hidden_dim`, `num_hidden_layers`
- `N_f`, `N_ic`

learning rate and optimizer

- for better accuracy.

[9]:
```
optimizer = optim.Adam(model.parameters(), lr=1e-3)

num_epochs = 5000
print_every = 500

for epoch in range(1, num_epochs + 1):
    optimizer.zero_grad()
    loss, loss_f, loss_ic = loss_function(model, N_f=2000, N_ic=200)
    loss.backward()
    optimizer.step()

    if epoch % print_every == 0:
        print(f"Epoch {epoch:5d} | Total: {loss.item():.4e} | "
              f"Physics: {loss_f.item():.4e} | IC: {loss_ic.item():.4e}")
```

```
Epoch   500 | Total: 9.4359e-03 | Physics: 9.3785e-03 | IC: 5.7323e-05
Epoch  1000 | Total: 6.3926e-03 | Physics: 6.3132e-03 | IC: 7.9448e-05
Epoch  1500 | Total: 5.8021e-03 | Physics: 5.7770e-03 | IC: 2.5056e-05
Epoch  2000 | Total: 4.4741e-03 | Physics: 4.4204e-03 | IC: 5.3736e-05
Epoch  2500 | Total: 3.8974e-03 | Physics: 3.8713e-03 | IC: 2.6090e-05
Epoch  3000 | Total: 3.9107e-03 | Physics: 3.6384e-03 | IC: 2.7225e-04
Epoch  3500 | Total: 3.0588e-03 | Physics: 3.0087e-03 | IC: 5.0133e-05
Epoch  4000 | Total: 2.9195e-03 | Physics: 2.8869e-03 | IC: 3.2650e-05
Epoch  4500 | Total: 2.6530e-03 | Physics: 2.6255e-03 | IC: 2.7504e-05
Epoch  5000 | Total: 1.9158e-03 | Physics: 1.7314e-03 | IC: 1.8444e-04
```

Using the trained PINN for inference

For a chosen damping ratio ξ* and a grid of z, evaluate x(z,ξ*):

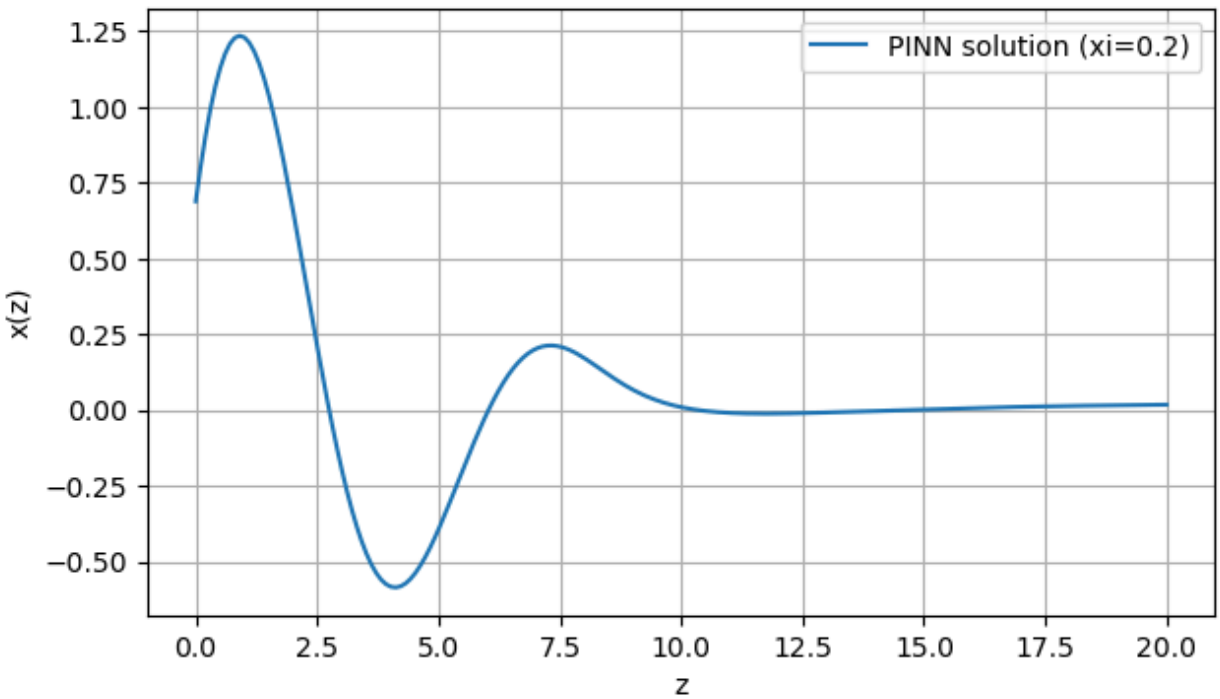
[10]:
```
import numpy as np

def predict(model, xi_value, num_points=200):
    model.eval()
    with torch.no_grad():
        z = torch.linspace(z_min, z_max, num_points).view(-1, 1).to(device)
        xi = torch.full_like(z, float(xi_value)).to(device)
        x_pred = model(z, xi)
    return z.cpu().numpy().flatten(), x_pred.cpu().numpy().flatten()
```

[11]:
```
import matplotlib.pyplot as plt

z_grid, x_grid = predict(model, xi_value=0.2)

plt.figure(figsize=(7,4))
plt.plot(z_grid, x_grid, label='PINN solution (xi=0.2)')
plt.xlabel('z')
plt.ylabel('x(z)')
plt.grid(True)
plt.legend()
plt.show()
```



2ND SOLUTION:

[2]:
```
!pip install torch matplotlib
```

```
Requirement already satisfied: torch in c:\users\acer\anaconda3\envs\sindy-env\lib\site-packages (2.10.0+cu126)
Requirement already satisfied: matplotlib in c:\users\acer\anaconda3\envs\sindy-env\lib\site-packages (3.10.6)
Requirement already satisfied: filelock in c:\users\acer\anaconda3\envs\sindy-env\lib\site-packages (from torch) (3.20.0)
Requirement already satisfied: typing-extensions>=4.10.0 in c:\users\acer\anaconda3\envs\sindy-env\lib\site-packages (from torch) (4.15.0)
Requirement already satisfied: sympy>=1.13.3 in c:\users\acer\anaconda3\envs\sindy-env\lib\site-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in c:\users\acer\anaconda3\envs\sindy-env\lib\site-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in c:\users\acer\anaconda3\envs\sindy-env\lib\site-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in c:\users\acer\anaconda3\envs\sindy-env\lib\site-packages (from torch) (2025.12.0)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\acer\anaconda3\envs\sindy-env\lib\site-packages (from matplotlib) (1.3.1)
Requirement already satisfied: cycler>=0.10 in c:\users\acer\anaconda3\envs\sindy-env\lib\site-packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\acer\anaconda3\envs\sindy-env\lib\site-packages (from matplotlib) (4.59.2)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\acer\anaconda3\envs\sindy-env\lib\site-packages (from matplotlib) (1.4.8)
Requirement already satisfied: numpy>=1.23 in c:\users\acer\anaconda3\envs\sindy-env\lib\site-packages (from matplotlib) (2.0.1)
Requirement already satisfied: packaging>=20.0 in c:\users\acer\anaconda3\envs\sindy-env\lib\site-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in c:\users\acer\anaconda3\envs\sindy-env\lib\site-packages (from matplotlib) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\acer\anaconda3\envs\sindy-env\lib\site-packages (from matplotlib) (3.2.0)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\acer\anaconda3\envs\sindy-env\lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\acer\anaconda3\envs\sindy-env\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in c:\users\acer\anaconda3\envs\sindy-env\lib\site-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\acer\anaconda3\envs\sindy-env\lib\site-packages (from jinja2->torch) (3.0.2)
```

[3]:
```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
device
```

[3]:
```
device(type='cpu')
```

[4]:
```python
class PINN(nn.Module):
    def __init__(self, in_dim=2, out_dim=1, hidden_dim=64, n_hidden=4):
        super().__init__()
        layers = []
        layers.append(nn.Linear(in_dim, hidden_dim))
        layers.append(nn.Tanh())
        for _ in range(n_hidden - 1):
            layers.append(nn.Linear(hidden_dim, hidden_dim))
            layers.append(nn.Tanh())
        layers.append(nn.Linear(hidden_dim, out_dim))
        self.net = nn.Sequential(*layers)

    def forward(self, z, xi):
        # z, xi: tensors of shape (N, 1)
        inp = torch.cat([z, xi], dim=1)
        return self.net(inp)
```

[5]:
```python
def pinn_residual(model, z, xi):
    """
    Compute residual of ODE:
    x'' + 2*xi*x' + x = 0
    """
    z = z.clone().detach().requires_grad_(True)
    xi = xi.clone().detach().requires_grad_(True)

    x = model(z, xi)                # (N, 1)
    dx_dz = torch.autograd.grad(
        x, z,
        grad_outputs=torch.ones_like(x),
        create_graph=True,
        retain_graph=True
    )[0]

    d2x_dz2 = torch.autograd.grad(
        dx_dz, z,
        grad_outputs=torch.ones_like(dx_dz),
        create_graph=True,
        retain_graph=True
    )[0]

    res = d2x_dz2 + 2.0 * xi * dx_dz + x
    return res
```

[6]:
```python
x0 = 0.7
v0 = 1.2

def ic_loss(model, n_ic=64):
    # Sample xi in [0.1, 0.4] for initial conditions
    xi_ic = torch.rand(n_ic, 1, device=device) * (0.4 - 0.1) + 0.1
    z_ic = torch.zeros_like(xi_ic, device=device)

    z_ic.requires_grad_(True)
    xi_ic.requires_grad_(True)

    x = model(z_ic, xi_ic)
    dx_dz = torch.autograd.grad(
        x, z_ic,
        grad_outputs=torch.ones_like(x),
        create_graph=True,
        retain_graph=True
    )[0]

    loss_x0 = torch.mean((x - x0) ** 2)
    loss_v0 = torch.mean((dx_dz - v0) ** 2)
    return loss_x0 + loss_v0
```

[7]:
```python
def physics_loss(model, n_collocation=1024):
    # Sample z in [0, 20], xi in [0.1, 0.4]
    z = torch.rand(n_collocation, 1, device=device) * 20.0
    xi = torch.rand(n_collocation, 1, device=device) * (0.4 - 0.1) + 0.1

    res = pinn_residual(model, z, xi)
    return torch.mean(res**2)
```

[8]:
```python
model = PINN().to(device)
optimizer = optim.Adam(model.parameters(), lr=1e-3)

n_epochs = 5000

for epoch in range(1, n_epochs + 1):
    optimizer.zero_grad()

    loss_pde = physics_loss(model, n_collocation=1024)
    loss_ic = ic_loss(model, n_ic=128)

    loss = loss_pde + loss_ic

    loss.backward()
    optimizer.step()

    if epoch % 500 == 0:
        print(f"Epoch {epoch}, Total Loss: {loss.item():.4e}, PDE: {loss_pde.item():.4e}, IC: {loss_ic.item():.4e}")
```

```
Epoch 500, Total Loss: 8.2844e-03, PDE: 8.2682e-03, IC: 1.6199e-05
Epoch 1000, Total Loss: 6.2434e-03, PDE: 5.8762e-03, IC: 3.6721e-04
Epoch 1500, Total Loss: 3.9367e-03, PDE: 3.9065e-03, IC: 3.0170e-05
Epoch 2000, Total Loss: 2.8859e-03, PDE: 2.7897e-03, IC: 9.6235e-05
Epoch 2500, Total Loss: 3.3760e-03, PDE: 3.0575e-03, IC: 3.1851e-04
Epoch 3000, Total Loss: 2.6018e-03, PDE: 2.4068e-03, IC: 1.9502e-04
Epoch 3500, Total Loss: 2.6902e-03, PDE: 2.3793e-03, IC: 3.1096e-04
Epoch 4000, Total Loss: 1.9128e-03, PDE: 1.8712e-03, IC: 4.1584e-05
Epoch 4500, Total Loss: 1.7272e-03, PDE: 1.6170e-03, IC: 1.1016e-04
Epoch 5000, Total Loss: 2.0232e-03, PDE: 1.9300e-03, IC: 9.3212e-05
```

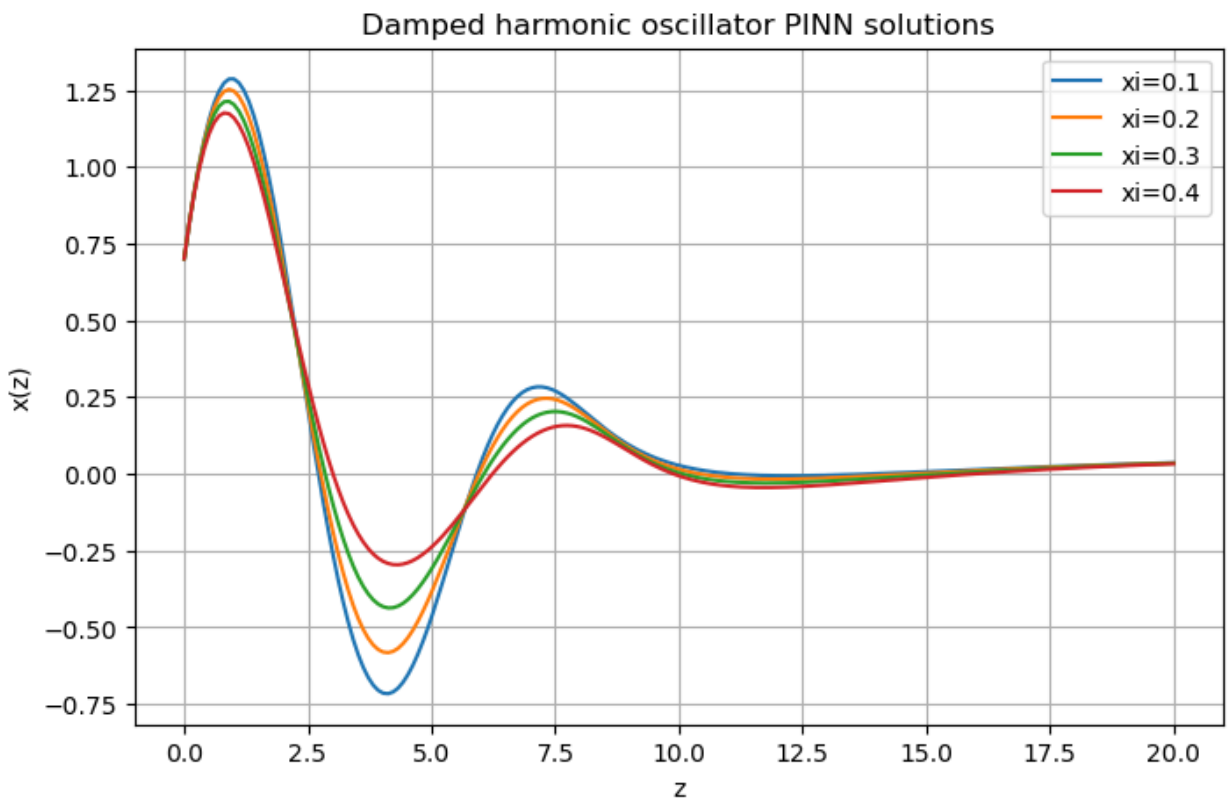
[9]:
```python
model.eval()

def predict_solution(xi_value, n_points=200):
    z = torch.linspace(0.0, 20.0, n_points, device=device).view(-1, 1)
    xi = torch.full_like(z, xi_value, device=device)
    with torch.no_grad():
        x = model(z, xi)
    return z.cpu().numpy().flatten(), x.cpu().numpy().flatten()

xis_to_plot = [0.1, 0.2, 0.3, 0.4]

plt.figure(figsize=(8, 5))
for xi_val in xis_to_plot:
    z_vals, x_vals = predict_solution(xi_val)
    plt.plot(z_vals, x_vals, label=f"xi={xi_val}")

plt.xlabel("z")
plt.ylabel("x(z)")
plt.title("Damped harmonic oscillator PINN solutions")
plt.legend()
plt.grid(True)
plt.show()
```


Damped harmonic oscillator PINN solutions
xi=0.1
xi=0.2
xi=0.3
xi=0.4
x(z)
z
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0